Figure 34.5 Using clustering with Quartz.NET allows horizontal scaling. Quartz.NET uses a database as a backing store, ensuring that only a single instance of the application handles a trigger at a time. This makes it possible to run multiple instances of your application to meet scalability requirements.

Quartz.NET relies on a persistent database for its clustering functionality. Quartz .NET stores descriptions of the jobs and triggers in the database, including when the trigger last fired. The locking features of the database ensure that only a single application can execute a task at a time.

TIP You can also enable persistence without enabling clustering, allowing the Quartz.NET scheduler to catch up with missed triggers.

Listing 34.13 shows how to enable persistence for Quartz.NET and how to enable clustering. This example stores data in a Microsoft SQL Server (or LocalDB) server, but Quartz.NET supports many other databases. This example uses the recommended values for enabling clustering and persistence as outlined in the documentation.

TIP The Quartz.NET documentation discusses many configuration setting controls for persistence. See the "Job Stores" documentation at http://mng.bz/PP0R. To use the recommended JSON serializer for persistence, you must also install the Quartz.Serialization.Json NuGet package.

Listing 34.13 Enabling persistence and clustering for Quartz.NET

```
using Quartz;

IHost host = Host.CreateDefaultBuilder(args)
    .ConfigureServices((hostContext, services) =>            ❶
    {
        var connectionString = Configuration                 ❷
            .GetConnectionString("DefaultConnection");       ❷
```